

Transaction Tracker System Design

1. Overview

A scalable system to:

- Upload financial transaction files (CSV/PDF)
 - Process and store transactions
 - Categorize transactions (manual + ML-based)
 - Provide low-latency dashboard analytics
 - Ensure idempotency, fault tolerance, and scalability
-

2. High-Level Architecture (HLD)

Components

1. Frontend (React)

2. Upload transactions
3. Dashboard (charts, categories)
4. Edit categories

5. API Gateway / Backend (Spring Boot)

6. Authenticated APIs
7. Upload endpoint
8. Dashboard APIs

9. Ingestion Service

10. Accepts file upload
11. Stores metadata
12. Pushes event to queue

13. Message Queue (Kafka/RabbitMQ)

14. Decouples ingestion from processing

15. Enables retry & scalability

16. Processing Service

17. Reads file in streaming fashion

18. Parses transactions

19. Inserts into DB

20. Publishes events

21. Transaction DB (Postgres/Oracle)

22. Source of truth (OLTP)

23. Aggregation Service

24. Consumes transaction events

25. Builds precomputed aggregates

26. Read Store (Redis / Aggregated DB)

27. Stores: USER_ID | MONTH | CATEGORY | TOTAL

28. Notification Service

29. Sends upload status updates

30. ML Service (Optional)

31. Predicts categories

3. Data Flow

Upload Flow

1. User uploads file → `/upload`
2. Backend:
3. Generates `fileHash`
4. Inserts into `uploads` table (IN_PROGRESS)
5. Pushes message to Kafka

6. Processing Service:
 7. Reads file (streaming)
 8. Inserts transactions
 9. Publishes transaction events
 10. Aggregation Service updates read store
 11. Upload marked as COMPLETE
 12. Notification sent
-

Dashboard Flow (Read Path)

1. User opens dashboard
 2. API fetches from:
 3. Redis / Aggregated DB
 4. Response time <200ms
-

4. Database Design (LLD)

uploads table

Column	Type
id	UUID
user_id	UUID
file_name	VARCHAR
file_hash	VARCHAR
status	ENUM
created_at	TIMESTAMP

Index:

- UNIQUE(file_hash, user_id)
-

transactions table

Column	Type
id	UUID
user_id	UUID
amount	DECIMAL
category	VARCHAR
timestamp	TIMESTAMP
merchant	VARCHAR

Indexes:

- (user_id, timestamp)
 - (user_id, category)
-

aggregates table

Column	Type
user_id	UUID
month	DATE
category	VARCHAR
total_amount	DECIMAL

Index:

- (user_id, month)
-

5. Key Design Decisions

5.1 Idempotency

- Generate `fileHash`
 - Before processing:
 - Check uploads table
 - Use: `UNIQUE(file_hash, user_id)`
-

5.2 File Processing

- Stream file (avoid loading in memory)
 - Batch inserts (e.g., 500 records)
-

5.3 Retry Mechanism

- Failed messages → Retry Queue
 - Retry N times
 - Store in `replay_table`
-

5.4 Read Optimization (CQRS)

- Write → transactions DB
 - Read → aggregates DB / Redis
-

5.5 Caching

- Redis for dashboard
 - TTL: 5-15 mins
 - Cache key: `dashboard:userId`
-

5.6 Consistency Model

- Eventual consistency (acceptable)
 - Slight delay in dashboard updates
-

6. API Design

Upload API

POST /upload

Response: { "uploadId": "123", "status": "IN_PROGRESS" }

Dashboard API

GET /dashboard

Response: { "monthlySpend": [...], "categoryBreakdown": [...] }

Update Category

PUT /transaction/{id}/category

7. Scaling Strategy

Horizontal Scaling

- Stateless APIs → scale via load balancer
 - Kafka partitions → parallel consumers
-

DB Scaling

- Read replicas for dashboard queries
 - Partition transactions table by user_id or date
-

Cache Scaling

- Redis cluster
-

8. Multi-Region Design

- Deploy services in multiple regions
- Use:
 - Geo-routing (Route53 / CloudFront)
- DB:
 - Primary + read replicas across regions
- Eventually:
 - Multi-master (if needed)

Trade-off:

- Eventual consistency across regions
-

9. Security Design

Data Security

- Encryption at rest (AES-256)
 - Encryption in transit (TLS)
-

Authentication

- OAuth2 / JWT
-

Authorization

- User-level data isolation
 - Row-level security
-

Sensitive Data

- Mask PII
 - Avoid logging sensitive info
-

10. Failure Handling

Failure	Solution
Processing fails	Retry queue
Duplicate upload	fileHash check
DB slow	Read replicas + indexing
Cache miss spike	Pre-warm cache

11. Monitoring & Observability

- Metrics: Grafana
- Logs: ELK / Splunk
- Alerts:
 - High latency
 - Queue backlog
 - DB slow queries

12. Performance Targets

Metric	Target
Upload API	< 500ms
Dashboard API	< 200ms
File Processing	Async

13. Future Enhancements

- ML-based categorization
- Real-time streaming dashboard
- Budget alerts
- Fraud detection

14. Tech Stack

- Backend: Spring Boot
- Frontend: React
- Queue: Kafka
- DB: Postgres / Oracle
- Cache: Redis
- Cloud: AWS (EC2, S3, CloudFront)

Final Note

This system follows:

- Event-driven architecture
- CQRS pattern
- Scalable & fault-tolerant design

Ready for production-scale systems.